ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino · Submit as PDF · Max 10 MB

Project Title	FreshIQ - Rotating-Tray Food Freshness Inspector
Team Name	[TO COMPLETE]
Registration / Team ID	[TO COMPLETE]
Contest Track	[TO COMPLETE] (see contest tracks)
Institution & City	[TO COMPLETE]

Team Members (1–4)

Name	Email	Role
Team Leader	[TO COMPLETE]	[TO COMPLETE]
Member 2 (optional)	[TO COMPLETE]	[TO COMPLETE]
Member 3 (optional)	[TO COMPLETE]	[TO COMPLETE]
Member 4 (optional)	[TO COMPLETE]	[TO COMPLETE]

1. Project Overview

Problem Statement

Food spoils in the home and in small retail mostly because nobody notices in time. Produce is checked by eye, irregularly, and usually only after one item has already turned and started affecting the fruit sitting next to it. The people this hits are ordinary households, hostel and canteen kitchens, and small grocery sellers - none of whom have any instrument for it, and all of whom throw away food that was edible a day earlier.

FreshIQ automates the check. Food sits in a four-compartment rotating tray; the device turns each compartment under a fixed camera, photographs it, and classifies what it sees as fresh or spoiled. The result is a dashboard that says which compartment needs attention, so the decision is made from a daily record rather than from memory.

How Your Project Works

The user opens the FreshIQ dashboard on a phone or laptop on the same Wi-Fi and presses Start scan. The scan controller on the UNO Q's Linux side asks the on-board microcontroller to turn the tray exactly one compartment - 50 motor steps - and waits for the microcontroller to

confirm the motion has finished. A short settle delay follows, because a 200 mm loaded platter is still swinging when the motor stops.

Only then does the camera capture. The frame passes a quality gate that rejects blurred, dark, overexposed or multi-object images with a named reason instead of guessing at them. Frames that pass go to a MobileNetV2 model with two output heads, running on-device through LiteRT: one head names the food, the other judges its condition. The scan controller attaches the compartment number - which only it knows, never the model - and appends the result to a local JSON Lines history.
The tray then advances to the next compartment and the cycle repeats until all four have been inspected. Every result is shown live on the dashboard as a compartment grid. Nothing leaves the device: inference, storage and the web interface all run on the UNO Q.

Why Arduino UNO Q?

FreshIQ needs two things that normally pull in opposite directions: enough compute to run a convolutional model on a camera frame, and hard real-time timing to step a motor without losing steps. The UNO Q gives both on one board, and that is the reason it was chosen rather than a convenience.

The Qualcomm Linux processor runs the TFLite model, the OpenCV pipeline, the HTTP server and the dashboard. The STM32U585 microcontroller generates the step pulses. That split matters: Linux is not a real-time operating system, and a scheduler hiccup inside a Python stepping loop produces missed steps and a tray that silently drifts out of alignment - a fault this rig cannot detect, because it has no encoder. Putting the pulse train on the microcontroller removes the problem entirely.
On-device inference also removes the network from the critical path. There is no cloud call, no API key and no round-trip latency; the device works with the Wi-Fi off, and no photograph of the user's kitchen ever leaves the board. For a food appliance that runs unattended, that is a privacy and reliability property, not just a performance one.

---

2. Components Used (BOM)

List every component. The Arduino UNO Q purchase proof must be uploaded separately with your submission.

Component	Qty
Arduino UNO Q (4 GB) - Linux MPU + STM32U585 MCU	1

NEMA 17 stepper motor, 17HS4401 class (1.8 deg, 200 steps/rev)	1
A4988 stepper driver carrier	1
USB camera, 1/4 in CMOS, UVC-compliant	1
12 V DC supply, 3 A / 36 W, 5.5 x 2.1 mm centre-positive	1
White LED strip, 12 V (constant illumination for the camera)	~0.5 m
Electrolytic capacitor 100 uF / 25 V (across VMOT - mandatory)	1
Lazy-susan turntable bearing, 4 in square	1
Motor coupling hub, 5 mm bore, 6-hole flange	1
3D-printed enclosure, deck, pedestal, tray carrier + food insert, camera gantry	10 parts
M3 / M4 / #8 fasteners and 4 mm dowel pins	assorted

3. System Architecture & Circuit

Step-by-Step Workflow

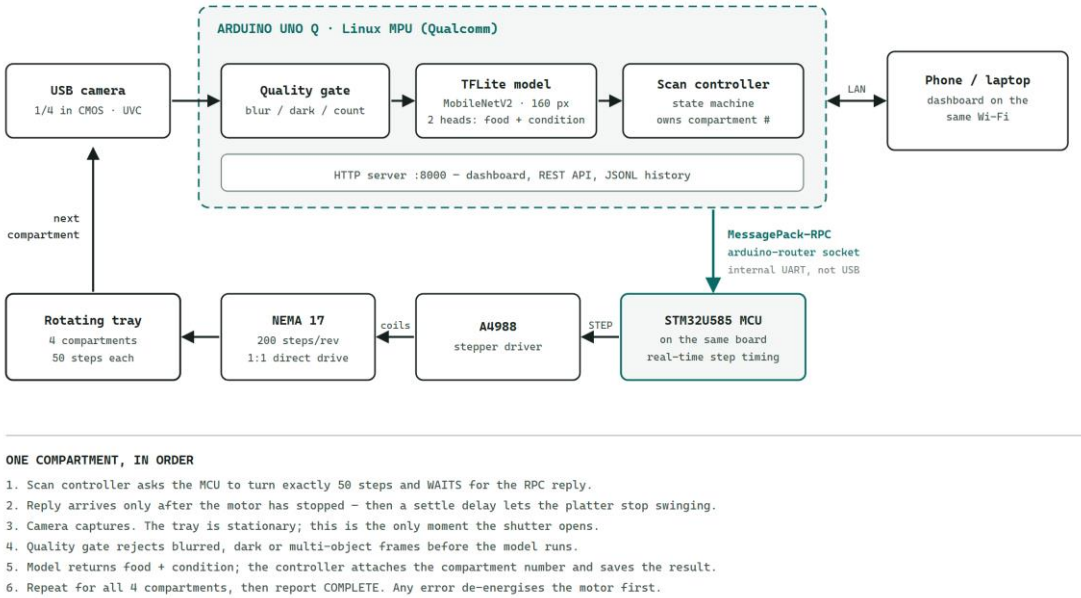
One full inspection cycle, in the order it actually happens:

1. User presses Start scan on the dashboard (phone or laptop, same Wi-Fi).
2. Scan controller (Linux) calls tray_rotate(50) on the microcontroller over MessagePack-RPC and blocks until it replies.
3. Microcontroller ramps the A4988 up and down over 50 steps, de-energises the driver, and only then returns - so the reply itself is the proof that motion ended.
4. Settle delay (1.2 s, configurable) lets the platter stop oscillating on its lazy-susan bearing.
5. Camera captures. This is the only moment the shutter opens, and the tray is stationary throughout.
6. Quality gate checks focus, brightness and object count; unusable frames are rejected with a reason and never reach the model.
7. MobileNetV2 (two heads) returns the food identity and its condition with confidences.
8. Scan controller attaches the compartment number, writes the result to the JSONL history and optionally saves the image.
9. Dashboard updates that compartment's card live.
10. Repeat for all four compartments, then report COMPLETE. On any error the motor is de-energised before anything else.

Block Diagram & Circuit Schematic

Paste your block diagram and circuit/wiring diagram below. Hand-drawn photos, Fritzing, KiCad — all accepted. Label pin connections.

System block diagram



ONE COMPARTMENT, IN ORDER

1. Scan controller asks the MCU to turn exactly 50 steps and WAITS for the RPC reply.
2. Reply arrives only after the motor has stopped — then a settle delay lets the platter stop swinging.
3. Camera captures. The tray is stationary; this is the only moment the shutter opens.
4. Quality gate rejects blurred, dark or multi-object frames before the model runs.
5. Model returns food + condition; the controller attaches the compartment number and saves the result.
6. Repeat for all 4 compartments, then report COMPLETE. Any error de-energises the motor first.

Figure 1 - Data flow. The camera feeds the Linux processor; the scan controller drives the on-board microcontroller over the Arduino Router's RPC socket; the tray returns the next compartment to the camera.

Circuit schematic

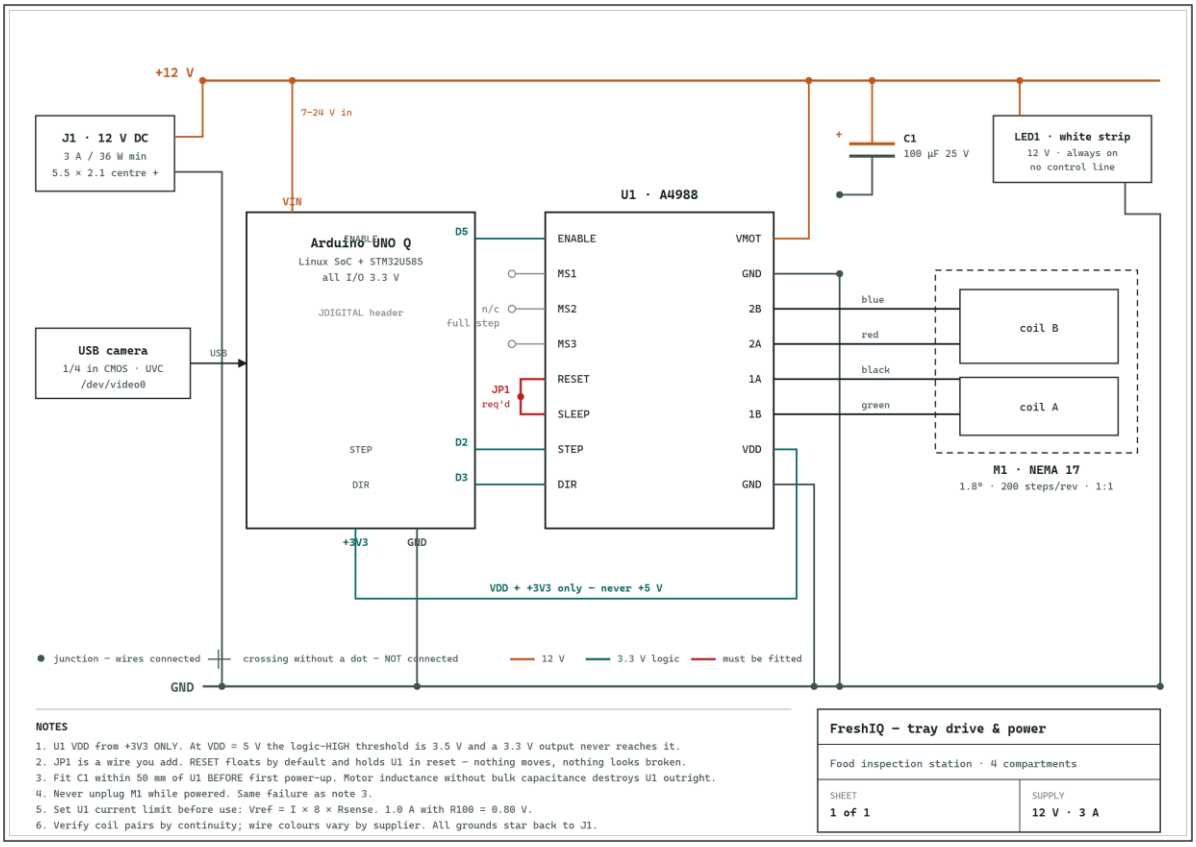


Figure 2 - Wiring. STEP = D2, DIR = D3, ENABLE = D5 (active low). A4988 VDD is taken from +3V3, never +5 V: at VDD = 5 V the driver's logic-HIGH threshold is 3.5 V and the UNO Q's 3.3 V outputs never reach it. RESET and SLEEP must be jumpered (JP1), and C1 fitted before first power-up.

4. AI / ML Model Details

If your project does not use AI/ML, write "Not Applicable" — but note that Physical AI projects score higher on Innovation.

Model Used	MobileNetV2 (alpha 1.0), two output heads, 160 x 160 RGB input, exported to TensorFlow Lite
Training Platform	TensorFlow / Keras on a laptop; inference on the UNO Q via LiteRT (ai-edge-litert). No TensorFlow is installed on the board.
Accuracy	Condition 99.92%, food identity 99.86%, spoiled recall 99.84% on 13,515 held-out test photos. See limitations below.
Dataset	Kaggle food-freshness-dataset (ulnnproject, Apache 2.0). 8 foods x 2 conditions: apple, banana, cucumber, mango, okra, orange, potato, tomato / fresh, spoiled.

Brief Description & Limitations

The network is a single MobileNetV2 backbone with two classification heads. One head has eight outputs and names the food; the other has two and judges fresh against spoiled. Splitting them matters because the two questions have different visual evidence, and because the food identity lets the system apply the right expectations - a brown banana and a brown potato are not the same finding.

Limitations, stated plainly:
The reported accuracy was measured on the public dataset, which is web photographs of supermarket produce - not photographs taken by this rig, under this lighting, at this angle. Real-world accuracy on the device will be lower than 99%, and we treat the figure as a training-time benchmark rather than a field claim. The dashboard includes a data-collection panel specifically to close that gap by fine-tuning on rig photographs.
The model is a classifier, not an object detector: it assumes one food item per compartment. Two different fruits in one compartment produce a multiple_objects rejection from the quality gate rather than two answers. It also cannot say "I have not seen this" - a ninth food is forced into one of the eight classes it knows.
The percentages are classification confidence, not a measurement of food safety. "Fresh classification: 96% confidence" does not mean the food is 96% fresh, and the system judges only the surface facing the camera - a fruit can look perfect and be rotten inside. FreshIQ is a prioritisation aid, not a safety instrument.

---

5. Code Structure

The project separates hardware, AI and orchestration so each can be tested alone. The AI package never touches a motor, and the motor code never imports the model.

config/freshiq.json
Everything about this particular rig: compartment count, motor driver, camera index, settle delays, image retention. Validated at startup - it refuses to run if, for example, the step count does not divide evenly into the compartments.
hardware/motor.py
TrayMotor.move_to(n) turns the tray forward to compartment n and blocks until it arrives. Three interchangeable drivers sit behind it: RouterBridgeTrayMotor (the UNO Q's on-board MCU), SerialTrayMotor (an external board) and SimulatedTrayMotor (no hardware).
hardware/router_bridge.py
MessagePack-RPC client for the Arduino Router. call() sends a request over /var/run/arduino-router.sock and returns the microcontroller's reply.
backend/scan.py
The scan state machine: IDLE - MOVING - STABILIZING - CAPTURING - ANALYZING - SAVING - COMPLETE, with ERROR and SAFE_STOP branches. It owns the compartment index and calls the AI once per compartment. Every exit path de-energises the motor.
ai/quality_check.py, ai/multihead.py, ai/aggregation.py
Reject unusable frames; run the two-head TFLite model; combine several views of one item using worst-view-wins.
backend/server.py
A standard-library HTTP server - no Flask, no FastAPI - serving the REST API and the dashboard from one process. Fewer dependencies is fewer things to fail on an embedded board.
firmware/freshiq_tray/sketch/sketch.ino
Runs on the UNO Q's microcontroller. Registers tray_rotate, tray_stop and tray_status with Bridge.provide_safe() and generates the ramped step pulses. It does not use Serial: on the UNO Q that port belongs to the Arduino Router.

GitHub Repository	[TO COMPLETE]
Demo Video Link	[TO COMPLETE]

6. Testing & Results

The system is covered by 118 automated tests that run without any hardware attached, because the motor and camera sit behind interfaces that can be substituted.

What the tests cover
Model contract - the deployed .tflite is opened and its tensors are read, asserting the input is 160 x 160 and the two heads are 8 and 2 wide, and that these match labels.json. This catches the failure where a model and its label file drift apart and the system confidently mislabels everything.
Scan cycle - a full four-compartment run end to end, asserting each compartment gets its own result, that the compartment number comes from the controller and not the model, and that no capture ever happens without a completed move before it.
Failure paths - camera returning no frame, cancellation mid-scan, invalid compartment, driver fault. Each asserts the motor ends de-energised.

Motor RPC - the MessagePack framing, message-ID matching and error propagation are exercised against a stand-in router, including a full 4 x 50 = 200-step revolution.

Measured results

Model, held-out test set: condition 99.92%, food 99.86%, spoiled recall 99.84% across 13,515 photos.

Inference: the two-head model is 2.7 MB and runs under LiteRT with the XNNPACK delegate; the model is loaded once at startup and reused for every compartment.

Scan timing: roughly 2-3 s per compartment, dominated by the 1.2 s mechanical settle delay rather than by inference.

Mechanical: all 10 printed parts verified as single closed manifold solids that fit a 256 mm print bed; each tray compartment provides 70.2 cm2 of floor area.

Edge cases handled

Blurred, too dark, too bright, no food detected, food too far, camera obstructed and multiple objects are each rejected by name rather than guessed at. Missing model, missing camera and unavailable tray each degrade to a clearly reported state instead of crashing the server.

[TO COMPLETE] Add your own on-hardware figures here after the first full physical run: measured cycle time, and how many of N real fruits were classified correctly under your lighting. Bench figures from the public dataset are not a substitute for that.

Project Images (2–3)

Mechanical design - parametric CAD, built and dimension-checked in Fusion 360:

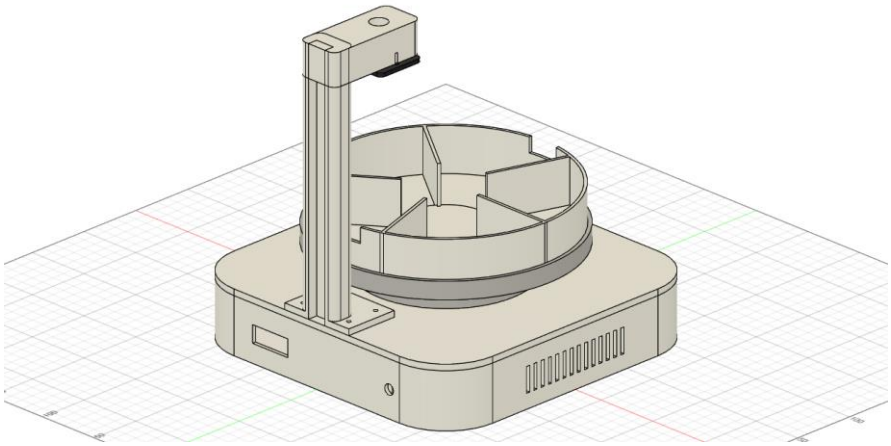


Figure 3 - Assembled: 250 x 250 x 275 mm. Four-compartment tray on a lazy-susan bearing, camera on the gantry, electronics in the base.

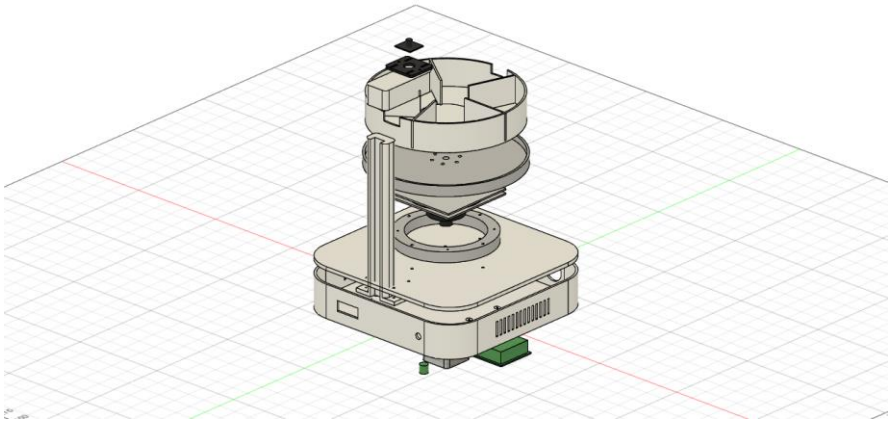


Figure 4 - Exploded: food insert, driven carrier, bearing, coupling hub, pedestal, deck, NEMA 17 and base.

[TO COMPLETE] Replace or supplement these with 2-3 photographs of the assembled physical rig running a scan, plus a screenshot of the dashboard mid-scan.

7. Challenges, Learnings & Future Improvements

Challenges Faced

The UNO Q is not a Linux box with an Arduino plugged into it.
The first working build drove the stepper over /dev/ttyACM0, which is how an external Arduino appears. On the UNO Q that device does not exist. Both processors are on one board, joined by an internal UART at /dev/ttyHS1 that the arduino-router service owns exclusively - Arduino's documentation says in as many words not to open it yourself. The fix was to replace the serial transport with a MessagePack-RPC client to the router's socket, and to rewrite the sketch to register handlers with Bridge.provide_safe() instead of reading Serial. Diagnosing this needed the board's own process list, not guesswork.
Averaging four different foods into one answer.
The AI layer aggregates several views into a single verdict using worst-view-wins, which is right for four photographs of one tomato and wrong for four compartments holding four different foods - it would blend a fresh apple and a rotten banana. The loop had to be inverted so the scan controller calls the AI once per compartment. The AI package itself was not modified.
Photographing a tray that had not finished moving.
The motor reporting that it has stopped is not the same as a 200 mm loaded platter having stopped. The original code wrote a rotate command and slept for a fixed 0.8 s. Now the microcontroller replies only after motion ends, and a separate configurable settle delay runs on top of that.
A driver that half-works is worse than one that does not work.
The A4988's logic-HIGH threshold is about 0.7 x VDD. Powered from 5 V that is 3.5 V, and the UNO Q's outputs only reach 3.3 V - so the motor twitches, or works until it warms up. VDD must come from +3V3. Two related traps are documented on the schematic: RESET floats and holds

the chip in reset unless jumpered to SLEEP, and omitting the bulk capacitor across VMOT destroys the driver on first power-up.

Making it printable.

The tray started at 240 mm, which forced a 300 mm base that no 256 mm printer can produce. Resizing and moving to four 90-degree compartments raised each compartment from 47.5 cm2 to 70.2 cm2 while letting every part print whole.

What You Learned & What's Next

We built a complete edge device: a parametric CAD enclosure, a printed four-compartment rotating tray, a stepper drive commanded from Linux through the UNO Q's on-board microcontroller, an on-device two-head vision model, a scan state machine and a dashboard - all running from one command on the board itself.

What we learned

Read the board's architecture before writing its driver. Most of the debugging time went on an assumption - that a microcontroller means a USB serial port - that was never true for this hardware.

A benchmark number is not a field number. 99.9% on web photographs says very little about performance under one particular kitchen light, and reporting it as though it did would be the easiest way to mislead someone.

Test what you actually measure. Our dimension tests initially checked only heights, and a bug where the camera arm did not physically reach over the camera passed cleanly. Three further bugs were found only by running the real system rather than the unit tests.

What is next

1. Fine-tune the model on photographs from this rig, under this lighting, to close the domain gap. The data-collection panel in the dashboard exists for exactly this.
2. Add a home sensor. The tray currently has no position feedback, so absolute position is unknowable and a skipped step is undetectable. A single optical switch would make the compartment index self-correcting.
3. Track each compartment over time rather than reporting a single moment, so the device can say which item is degrading fastest and should be eaten first - which is the question a user actually has.

---

8. Declaration

We confirm this is our original, unpublished work. The Arduino® UNO™ Q is the primary board in this project. All team members have reviewed and agree to this report.

Date	[TO COMPLETE]